Niezawodność systemów informatycznych

Raport numer 1 (P1)
Pomiar niezawodności testowego oprogramowania

Trudov Mykhailo, Mykola Blavitskyi, Oleh Stasiuk
Zespół G

12 października 2026

SUT

System Under Test to **system zarządzania kontem bankowym** (wariant zespołu G). Udostępnia operacje: utworzenie konta, wpłata, wypłata, przelew wewnętrzny, naliczanie odsetek, historia, wyciąg miesięczny oraz przelew zewnętrzny przez symulowany procesor płatności. Numery kont są 10-cyfrowe; w kodzie istnieje suma kontrolna (`_numer_poprawny`), ale `utworz_konto` jej nie stosuje. SUT nie był poprawiany: zebrany pomiar to punkt odniesienia na semestr.

Repozytorium

Link do repozytorium: https://github.com/SadAndAlone/SUT_zespolG_MT_OS_MB

Znalezione awarie

Klasyfikacja dotkliwości jest stała w zespole na cały semestr. Nie wliczamy oczekiwanej walidacji (`ValueError` przy ujemnej kwocie itd.) — to poprawne zachowanie, nie *failure*.

Typ	Dotkliwość	Opis
złe dane wyjściowe	poważna	`wplac` dwukrotnie zapisuje historię: saldo rośnie raz, wyciąg zawyża wpłaty.
brak walidacji	poważna	Konto z błędną cyfrą kontrolną zostaje utworzone (`_numer_poprawny` nieużywane).
brak wyjątku	poważna	Wpłata na konto zablokowane jest przyjmowana.
brak wyjątku	poważna	Przelew *na* konto zablokowane przechodzi (sprawdzany jest tylko nadawca).
wyjątek `InvalidOperation`	poważna	`wyplac(None)` nie sprawdza typu kwoty (w przeciwieństwie do `wplac`).
zły komunikat	drobna	Duplikat konta: komunikat `Konto {numer} juz istnieje` (nawiasy w f-stringu).
zły komunikat	drobna	Przelew z zablokowanego: literał `{numer_z}` zamiast numeru.

Osiem obserwowanych awarii pochodzi z **siedmiu** unikalnych usterek: podwójna historia ujawnia się i przy `wplac`, i przy wyciągu miesięcznym (jedna usterka, dwa *failure*).

Wstępne wskaźniki (sesja eksploracyjna)

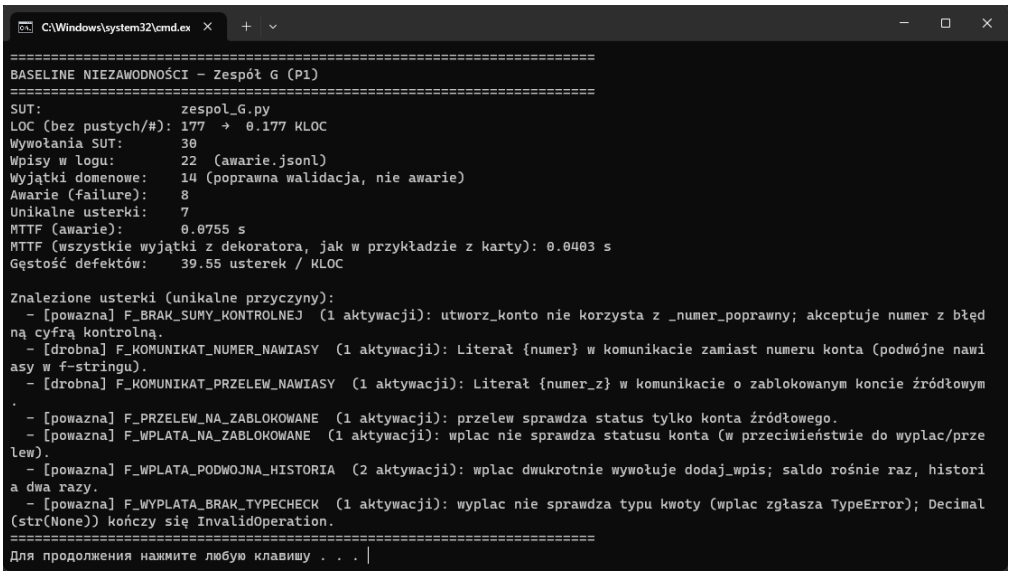
- liczba wywołań SUT: **30**
- liczba awarii (*failure*): **8**

- unikalne usterki (*fault*): **7**
- LOC SUT (bez pustych i komentarzy): **177** → 0,177 KLOC
- MTTF: 0,075 **s** (średni odstęp między kolejnymi awariami od startu sesji; wywołania laboratoryjne są gęste, więc wartość jest rzędu ułamka sekundy — baseline do porównań, nie prognoza produkcyjna)
- gęstość defektów: 7/0,177 ≈ 39,55 **usterek / KLOC**

Dane surowe: `awarie.jsonl` (timestamp, funkcja, argumenty, typ, komunikat, dotkliwość). Logger nie połyka wyjątków.

Wniosek — profil operacyjny

Najbardziej ryzykowne są ścieżki codziennego ruchu — **wpłata i przelew**: usterka w `wplac` psuje historię i wyciąg, a przelew na zablokowane konto oraz wpłata na zablokowane konto obchodzą blokadę na operacjach, które w banku wykonuje większość użytkowników; rzadki przelew zewnętrzny (kruchy procesor) zostawiamy na blok P5 i nie wliczamy go do baseline.



1 Pytania kontrolne

1. **Czym różni się fault od failure? Podaj przykład usterki, która nigdy nie ujawni się jako awaria.**

   Fault (usterka) to defekt w kodzie lub projekcie. *Error* to błędny stan wewnętrzny po aktywacji usterki. *Failure* (awaria) to już obserwowalne odstępstwo od specyfikacji. Łańcuch: fault → error → failure. Usterka nie musi dać awarii, jeśli nigdy nie zostanie wykonana albo błąd stanu zostanie zamaskowany. Przykład: literówka w warunku wewnątrz `nalicz_odsetki` dla `miesiace > 120`, podczas gdy w profilu operacyjnym bank nigdy nie nalicza odsetek na taki horyzont — usterka istnieje w kodzie, ale nie staje się *failure*.

2. **Dlaczego dwa programy z taką samą liczbą usterek mogą mieć różną niezawodność?**

   Niezawodność zależy od tego, *jak często* usterki są aktywowane przy typowym użytkowaniu (profil operacyjny), a nie od samej liczby *faults*. Jedna usterka na ścieżce 80% ruchu psuje odczuwaną niezawodność bardziej niż pięć usterek w martwym kodzie.

3. **Co oznacza dostępność na poziomie 99,9% w godzinach przestoju rocznie?**

   Rok ma $365 \times 24 = 8760$ godzin. Przy $A = 99{,}9\%$ na przestój zostaje 0,1%, czyli $0{,}001 \times 8760 = 8{,}76$ godziny przestoju rocznie.

4. **Dlaczego profil operacyjny jest istotny przy planowaniu testów?**

   Opisuje, które funkcje są wołane i w jakich proporcjach. Testy powinny odzwierciedlać rzeczywiste użycie, a nie dzielić czasu „po równo” na wszystkie metody. Inaczej przecenimy rzadkie ścieżki i niedoszacujemy awarii na głównej (jak w Ariane 5: ten sam kod, inny profil, inny skutek).

5. **Jakie dane musi zawierać wpis w logu awarii, żeby awaria była powtarzalna?**

   Znacznik czasu, wywoływana funkcja/moduł, rodzaj awarii, dotkliwość oraz **dane wejściowe** (argumenty), które ją wywołały — bez nich nie da się odtworzyć przypadku. Nasz wpis JSONL ma m.in. `timestamp`, `funkcja`, `args`, `typ_bledu`, `komunikat`, `severity`.

6. **Czym różni się awaria krytyczna od poważnej? Podaj przykład własny.**

   Krytyczna: system przestaje działać, dane giną/ulegają uszkodzeniu albo jest naruszenie bezpieczeństwa. Poważna: konkretna funkcja działa źle, ale system jako całość zostaje stabilny. Przykład własny (nasz SUT): *krytyczna* — przelew odejmuje kwotę u nadawcy i nie dopisuje jej u odbiorcy (pieniądze znikają); *poważna* — wpłata księguje saldo poprawnie, ale podwaja wpis w historii (wyciąg jest zły, środki nie giną).

7. **Mając cztery awarie w czasach 12 s, 47 s, 53 s, 118 s, pokaż jak policzyć MTTF i podaj wynik.**

   Odstępy od startu / między awariami: $\Delta_1 = 12$, $\Delta_2 = 47 - 12 = 35$, $\Delta_3 = 53 - 47 = 6$, $\Delta_4 = 118 - 53 = 65$.

   $$\mathrm{MTTF} = \frac{12 + 35 + 6 + 65}{4} = \frac{118}{4} = 29{,}5\,\mathrm{s}.$$